



Facultad de Ingeniería - Udelar

Práctico 1

Patrones de acceso a memoria

Manuel Roca

Belén Olivera

Nicolás Pereira

25 de marzo de 2026

1. Ejercicio 1 - Localidad espacial

Se utilizó un arreglo grande de 100MB para asegurar que no entre completamente en caché. Se midieron tiempos para acceso secuencial, aleatorio y con distintos saltos.

1.1. Acceso secuencial vs aleatorio

En el acceso secuencial se recorren posiciones contiguas del arreglo, mientras que en el acceso aleatorio se utiliza un orden mezclado previamente sin ningún orden específico. En la siguiente tabla exponemos los resultados obtenidos.

Iteración	Secuencial (s)	Aleatorio (s)
1	0.191777	0.475792
2	0.039597	0.465766
3	0.034483	0.473222
4	0.034487	0.463038
5	0.034497	0.520853
6	0.034464	0.474389
7	0.035131	0.466162
8	0.033975	0.462687
9	0.034541	0.475565
10	0.034150	0.463524
11	0.034435	0.472313
12	0.034452	0.464908
13	0.034645	0.468804
14	0.034522	0.459466
15	0.034411	0.468153
Promedio (2-15)	0.03495	0.47135

Cuadro 1: Tiempos de ejecución por iteración para acceso secuencial y aleatorio

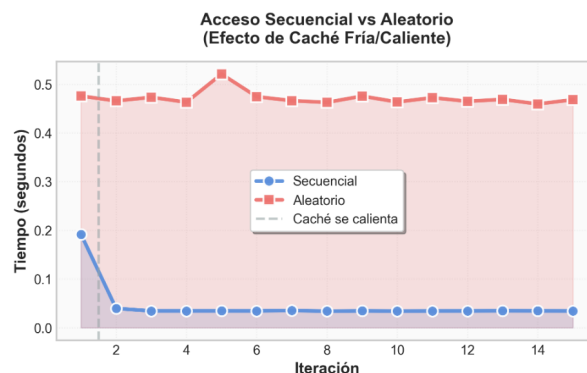


Figura 1: Tiempo de ejecución por iteración para acceso secuencial y aleatorio

Explicación:

Se observa que el acceso secuencial es significativamente más rápido que el acceso aleatorio. Además, la primera iteración resulta más lenta debido a que la caché está fría, mientras que en iteraciones posteriores el rendimiento mejora.

Esto se explica por la localidad espacial: cuando se accede a una posición de memoria, se carga en caché un bloque completo, por lo que los accesos a posiciones contiguas ya se encuentran

disponibles. En cambio, en el acceso aleatorio no hay (o es muy poca) reutilización de datos, lo que genera una mayor cantidad de caché misses.

Por otro lado, en el acceso secuencial el hardware puede aplicar técnicas de *prefetching*, adelantando la carga de datos antes de que sean necesarios, lo que también contribuye a mejorar el rendimiento.

1.2. Caso particular: saltos fijos

A modo de complemento decidimos probar el acceso aleatorio con distintos valores fijos de saltos. Accedemos a la misma matriz que en la parte 1 del ejercicio.

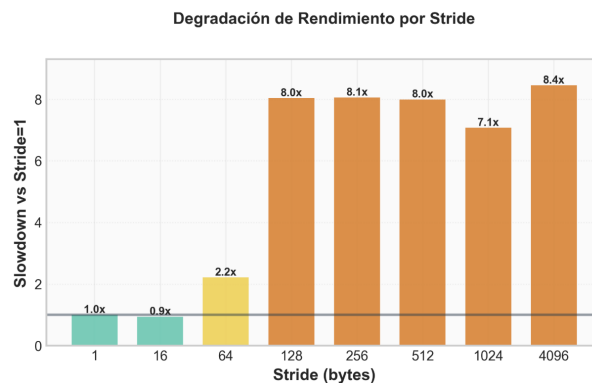


Figura 2: Rendimiento por tamaño de salto

Observaciones:

- Saltos chicos (1, 16) tiene buen rendimiento.
- Saltos cercano al tamaño de línea (64) empieza a degradar.
- Strides grandes (tamaño línea 128 o mayores) empeoran mucho el rendimiento.

Explicación:

Cuando el stride es chico, se aprovecha la localidad espacial. A medida que aumenta, se salta entre bloques de memoria, perdiendo eficiencia de caché.

- Saltos chicos (1, 16): Se realizan suficientes instrucciones por línea de caché, aprovechando la localidad espacial.
- Saltos cercanos al tamaño de línea (64): Se realizan pocas instrucciones (2) antes de reemplazar la línea de caché, perdiendo eficiencia.
- Strides grandes (128 o mayores): Por cada instrucción se reemplaza la línea de caché, saltando entre bloques de memoria. El reemplazo de línea de caché se convierte en el cuello de botella.

2. Ejercicio 2 - Multiplicación de matrices

Se compararon distintas implementaciones de multiplicación de matrices, enfocándose en cómo el acceso a memoria afecta el rendimiento, para las pruebas realizadas en 2.1 y 2.2 se utilizan

matrices de tamaño 2048×2048 de floats, de modo que el conjunto de trabajo no entre en la caché de último nivel (LLC) de 4MB.

2.1. Determinación del tamaño de bloque (BS)

Se probaron distintos tamaños de bloque.

BS	Tiempo (s)	MFLOPS	Observación
8	7.2373	2373.80	Óptimo
16	11.9773	1434.37	
32	12.3692	1388.93	
64	12.9461	1327.03	
128	12.4964	1374.79	
256	12.3062	1396.03	

Cuadro 2: Rendimiento de multiplicación de matrices (2048×2048) para distintos tamaños de bloque

Explicación:

El objetivo del blocking es lograr que los datos que se están usando en cada momento entren en caché, de forma de poder reutilizarlos y evitar accesos a memoria más lenta.

Como en la multiplicación intervienen tres matrices (A, B y C), en cada paso se están utilizando bloques de cada una. Por eso, una idea razonable es que el mejor tamaño de bloque sea el más grande posible que permita que los tres bloques entren al mismo tiempo en caché, de forma de maximizar la reutilización de los datos.

Si la caché de datos L1 es de $64KB$ ($65536Bytes$), el mejor BS esperado es de $BS*BS*4*3$, donde:

- $BS*BS$ cantidad de elementos del bloque
- 4 Bytes el tamaño de float en C/C++
- 3 bloques activos, uno de cada matriz (A, B y C)

Siguiendo esto, el valor de BS óptimo que esperamos es aproximado a $BS = 64$, sin embargo tras hacer las pruebas obtuvimos que $BS = 8$ es el óptimo. Este valor es menor al esperado teóricamente.

La cuenta anterior asume que toda la caché L1 está disponible para los bloques lo cual no es realista. En la práctica, la caché también guarda otras cosas como variables, el stack y el propio código, por lo que el espacio realmente disponible es menor.

Además se pueden generar conflictos por la asociatividad: distintos bloques pueden mapear a las mismas posiciones de caché y terminar reemplazándose entre sí, incluso si en teoría habría espacio suficiente. Al haber reemplazos en la cache L1 se generan accesos a niveles inferiores de la jerarquía de memoria (L2 o L3), las cuales tienen latencias significativamente mayores (entre 3 y 10 veces más lentas).

2.2. Comparación blocked vs IKJ

Se comparó la versión por bloques usando el tamaño $BS = 8$, que arrojó mejores resultados, con la variante lineal IKJ.

Variante	BS	Tiempo (s)	MFLOPS
blocked	8	7.8412	2190.96
ikj	-	0.7196	23874.27

Cuadro 3: Comparación entre versión blocked y variante lineal ikj

Explicación:

La variante IKJ accede a los datos de forma contigua en memoria, aprovechando el almacenamiento por filas de las matrices. En particular, para valores i, k fijos, con el loop j avanzando, se accede al dato $A[i][k]$ y luego los arreglos B y C se recorren por filas, haciendo buen uso de la localidad espacial.

La variante por bloques, en cambio, presenta el problema de que accede a la matriz B por columnas, lo que genera saltos en memoria y empeora el rendimiento de la caché.

Si bien usar bloques tiene el potencial de reducir los accesos a memoria principal el orden de recorrido dentro de cada bloque sigue siendo importante para el rendimiento por lo que lo ideal sería combinar ambas ideas: el particionamiento en bloques y un orden de acceso como el ikj que ayude con la localidad espacial.

2.3. Orden de loops

En esta parte se evaluó el rendimiento de distintas variantes de orden de loops para la multiplicación de matrices en varios tamaños.

Los resultados muestran que el orden de los loops tiene un impacto significativo en el rendimiento. La variante **IKJ** es consistentemente la más eficiente, seguida por **KIJ**, mientras que el resto presenta desempeños considerablemente menores.

En promedio:

- IKJ: ~ 26198 MFLOPS
- KIJ: ~ 21447 MFLOPS
- JKI: ~ 2502 MFLOPS
- IJK, JIK, KJI: ~ 1800 MFLOPS

Las diferencias se deben principalmente al patrón de acceso a memoria:

- **IKJ**: acceso secuencial a las matrices, maximizando la localidad espacial y el uso de caché.
- **KIJ**: comportamiento similar, A se recorre por columnas por lo que empeora el rendimiento respecto al caso anterior.
- **IJK y JIK**: acceso por columnas a la matriz B en el loop más interno lo que es más costoso porque se recorre muchas veces sin aprovechar localidad espacial, generando una mayor cantidad de cache misses.
- **JKI y KJI**: acceso por columnas en múltiples matrices, resultando en el peor rendimiento.

Impacto del tamaño de la matriz

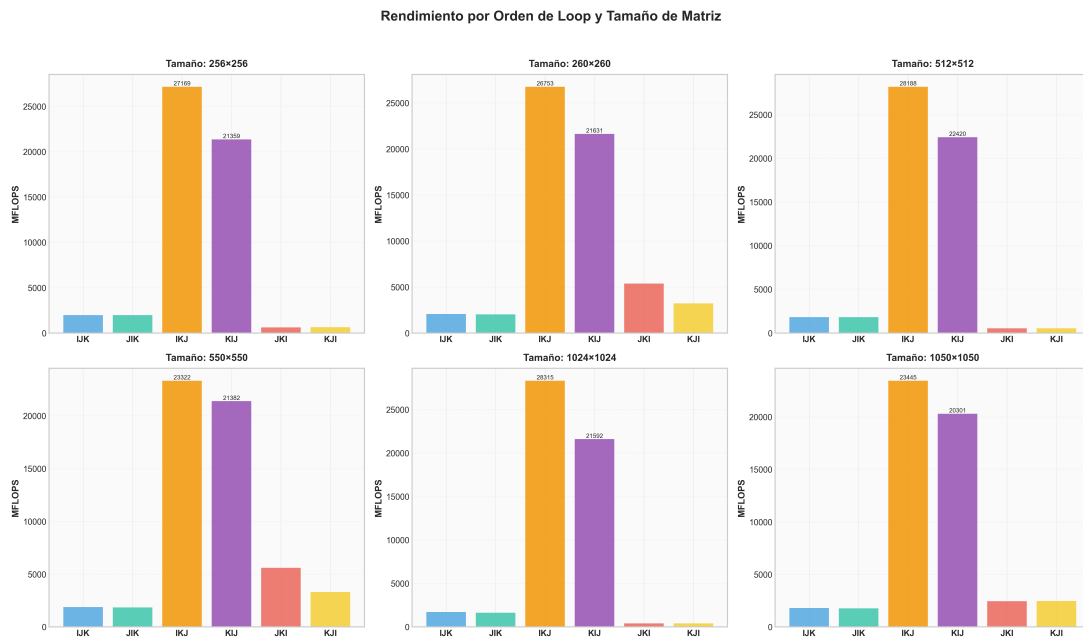


Figura 3: Rendimiento por orden de loop y tamaño de matriz

Al comparar por ejemplo el caso kji según si N es potencia de 2 o no, se observa una diferencia notable en el rendimiento: los casos donde N es potencia de 2 presentan el peor desempeño. Lo mismo sucede en el caso jki.

Creemos que esto se debe a que, en estos ordenes, A y C se recorren por columnas, por lo que el bucle interno sobre B accede a memoria con un patrón de potencia 2. Al calcular el conjunto de caché (caché set) donde caen estas direcciones, múltiples accesos terminan mapeando al mismo conjunto, provocando conflictos y degradando el rendimiento.

3. Consideraciones generales

3.1. Entorno de ejecución

Todas las pruebas expuestas en el documento fueron ejecutadas en una laptop con procesador Apple Silicon M2. De todos modos, también realizamos pruebas conectándonos por SSH a las máquinas PC Unix de Fing, con el objetivo de ejecutar el programa fijando un núcleo.

Comparamos el comportamiento tanto fijando un núcleo como sin hacerlo. En ambos casos, el rendimiento fue ligeramente inferior al de la máquina local, aunque se obtuvieron resultados consistentes.

3.2. Medicion del tiempo

La medición temporal se realizó con *gettimeofday()*, este tiempo incluye tanto el cómputo como las latencias de acceso a memoria, permitiendo evaluar cómo el blocking y el orden de loops afectan el aprovechamiento de la jerarquía de caché.

```
1 double benchmark_blocked(float *A, float *B, float *C, int N, int BS) {  
2     memset(C, 0, N * N * sizeof(float)); // limpiamos matriz C  
3     double start = get_time(); // inicio  
4     matrix_mult_blocked(A, B, C, N, BS); // multiplicacion de matrices  
5     double end = get_time(); // fin  
6     return end - start; // tiempo de ejecucion  
7 }
```